

Improving Distributional Similarity with Lessons Learned from Word Embeddings

Levy, Goldberg, and Dagan · TACL 2015

부제: word2vec은 왜 PPMI/SVD보다 빠르고 성능 좋은걸까?

2014년의 분위기

Word Embedding...

Count 기반

단어-문맥 동시출현 빈도 행렬
PPMI 계산 · SVD(저차원 축소)
오래된 방식이라는 인식

VS

Prediction 기반

(Positive 샘플과 Negative 샘플을 구별,
단어-문맥 관계를 신경망 학습
word2vec · SGNS
더 발전된 방식이라는 인식

Baroni et al.(2014) 의 '**Don't count, predict! ~**' 연구 논문: Prediction 모델이 여러 과제에서 count 모델보다 높은 성능
전반적으로 Prediction 모델이 본질적으로 우수하다는 인식

"왜 word2vec이 잘 작동하는가?"

2014년, 본 논문의 저자들이 의문을 가지고 한 연구를 수행.

“Word2Vec은 왜 속도도 빠르고 성능도 좋은걸까?”

그 결과를 논문(*Neural Word Embedding as Implicit Matrix Factorization*)으로 발표.

Finally, we can describe the matrix M that SGNS is factorizing:

$$M_{ij}^{\text{SGNS}} = W_i \cdot C_j = \vec{w}_i \cdot \vec{c}_j = \text{PMI}(w_i, c_j) - \log k$$

SGNS로 학습한 학습한 벡터(W&C)내적이 Shifted PMI와 같다는 걸 수학적으로 밝혀냄.

PMI는 카운트 기반 방법이므로, 결국 두가지 방법의 목적지가 같았던 것!!

기존 비교에서 달랐던 조건

알고리즘과 설정의 효과가 함께 측정됐다

Count기반 방법과 Prediction기반 방법의 결과가 수학적으로 같다.
그렇다면 왜? Prediction기반 방법이 성능이 더 좋게 나왔을까?

기존 연구들이 두가지를 동등한 조건에서 비교한적 없다.

기존 연구에서 SGNS가 여러 성능 개선 설정을 포함,
But, PPMI와 SVD는 상대적으로 기본적인 설정으로 평가됨.

기존 비교에서 달랐던 조건

알고리즘과 설정의 효과가 함께 측정됐다

설정	PPMI · SVD	SGNS
Dynamic context window	기본	적용
Frequent-word subsampling	기본	적용
Shift from negative sampling	미적용	적용
Context distribution smoothing	미적용	적용
SVD eigenvalue weighting	관습적 설정	해당 없음

저자들은 효과적인 설정을 다른 방법에도 옮겨 동일한 조건에서 다시 비교했다

과제에 따라 달라진 최적 방법

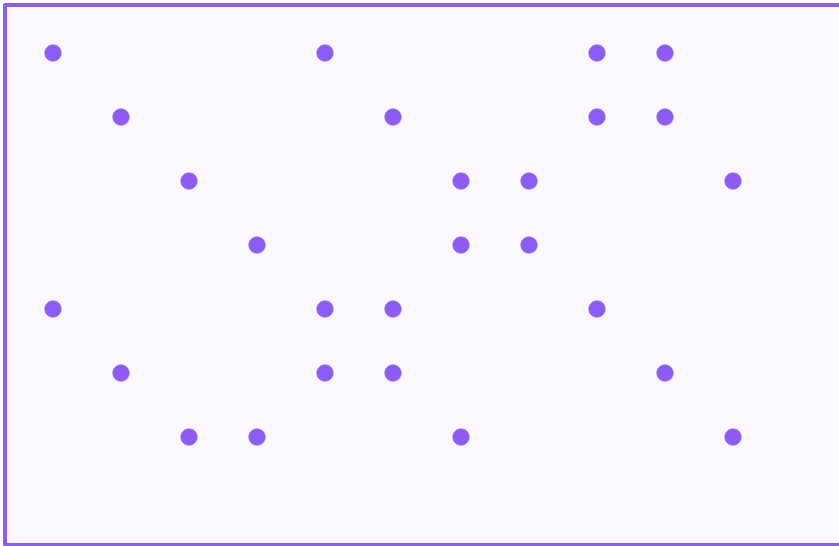
win	Method	WordSim Similarity	WordSim Relatedness	Bruni et al. MEN	Radinsky et al. M. Turk	Luong et al. Rare Words	Hill et al. SimLex	Google Add / Mul	MSR Add / Mul
2	PPMI	.732	.699	.744	.654	.457	.382	.552 / .677	.306 / .535
	SVD	.772	.671	.777	.647	.508	.425	.554 / .591	.408 / .468
	SGNS	.789	.675	.773	.661	.449	.433	.676 / .689	.617 / .644
	GloVe	.720	.605	.728	.606	.389	.388	.649 / .666	.540 / .591
5	PPMI	.732	.706	.738	.668	.442	.360	.518 / .649	.277 / .467
	SVD	.764	.679	.776	.639	.499	.416	.532 / .569	.369 / .424
	SGNS	.772	.690	.772	.663	.454	.403	.692 / .714	.605 / .645
	GloVe	.745	.617	.746	.631	.416	.389	.700 / .712	.541 / .599
10	PPMI	.735	.701	.741	.663	.235	.336	.532 / .605	.249 / .353
	SVD	.766	.681	.770	.628	.312	.419	.526 / .562	.356 / .406
	SGNS	.794	.700	.775	.678	.281	.422	.694 / .710	.520 / .557
	GloVe	.746	.643	.754	.616	.266	.375	.702 / .712	.463 / .519
10	SGNS-LS	.766	.681	.781	.689	.451	.414	.739 / .758	.690 / .729
	GloVe-LS	.678	.624	.752	.639	.361	.371	.732 / .750	.628 / .685

잘 튜닝한 SVD와 PPMI도 일부 과제에서 최고 성능을 기록했다

최적 설정은 vanilla 대비 평균 6%p 이상, 최대 15.7%p 향상

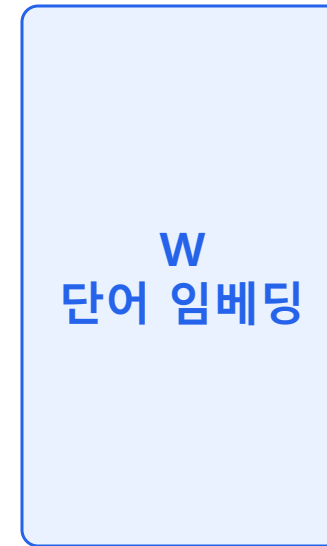
그럼에도 SGNS가 실용적인 이유

PPMI · SVD



거대한 단어 × 문맥 희소행렬

SGNS



관찰된 쌍과 negative sample만 처리

SGNS는 가장 빠르고 메모리·디스크 사용량이 가장 적은 robust baseline

결론

성능 차이를 만든 것은 무엇이었나

알고리즘

전처리, 학습 설정

하이퍼파라미터

같은 조건에서 비교하자 방법 간 격차는 작아졌고, 과제별 승자는 달라졌다

SGNS의 강점은 보편적인 성능 우위보다 안정성·효율성·확장성에 있다

감사합니다.